Passing arguments through

Often, in Ruby, we want to pass all the arguments from one method to another method. Since there are three different kinds of arguments that a Ruby method can take, simply doing passthroughs can feel kind of cumbersome…

```
def outer(*args, **kwargs, &block)
  other(*args, **kwargs, &block)
end
```

Ruby has some shortcuts if you aren't actually using the arguments in the method. For all three types, you can pass through the arguments of that particular kind by using the splat or ampersand without a variable name:

```
def outer(*)
  other(*)
end

def outer(**)
  other(**)
end

def outer(&)
  other(&)
end
```

If you are passing multiple types of variables, you can forward them all with ....

```
def outer(...)
  other(...)
end
```

You can even include leading positional arguments before the forwarding marker:

```
def outer(first, ...)
  other(first, ...)
end
```

Undefining a Method

The keyword `undef` allows you to undefine a method.

undef *name* | *symbol* ...

An undefined method still exists; it is simply marked as being undefined. If you undefine a method in a child class and then call that method on an instance of that child class, Ruby will immediately raise a `NoMethodError`—it will not look for the method in the child's parents.

Invoking a Method

‹*receiver.*›*name*‹ *arguments* › ‹ *{block}* ›
‹*receiver::*›*name*‹ *arguments* › ‹ *{block}* ›

arguments ← (‹ *arg* ›* ‹, *hashlist* › ‹ **array* › ‹ *&a_proc* ›)

block ← { *blockbody* } *or* do *blockbody* end